SUBMITTED TO: SIR SAMRAIZ

SUBMITTED BY: EMAN TAHIR

ROLLNO. 2023-BS-AI-015

SUBJECT: PROGRAMMING FOR AI

DEPARTMENT: ARTIFICIAL INTELLIGENCE

Contents

LAB-01

TASKS:

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal).

```
[7]: pin = "0000"
balance = 5000
tries = 0
while tries < 3:
    user_pin = input("Enter PIN: ")
    if user_pin == pin:
        print(f"Balance: PKR {balance}")
        amount = float(input("Withdraw amount: "))
        if amount <= balance:
            charge = amount * 0.02
            balance -= (amount + charge)
            print(f"Withdrawn: {amount}, Charge: {charge}")
            print(f"New Balance: PKR {balance}")
        else:
            print("Insufficient funds")
        break
    else:
        tries += 1
        print("Wrong PIN")
        if tries == 3:
            print("Card blocked")
```

```
Enter PIN:  1111
Wrong PIN
Enter PIN:  2222
Wrong PIN
Enter PIN:  3333
Wrong PIN
Card blocked
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

[57]:
```
items = {
    "apple": {"price": 100, "stock": 10},
    "banana": {"price": 50, "stock": 20},
    "mango": {"price": 150, "stock": 15}
}
cart_total = 0
item = input("Enter item name: ").lower()
qty = int(input("Enter quantity: "))
if item in items and qty <= items[item]["stock"]:
    price = items[item]["price"] * qty
    items[item]["stock"] -= qty
    cart_total += price
else:
    print("Not enough stock or item not found!")
if cart_total >= 1000:
    discount = 0.20
elif cart_total >= 500:
    discount = 0.15
elif cart_total >= 200:
    discount = 0.10
else:
    discount = 0
cart_total -= cart_total * discount
cart_total += cart_total * 0.05
print(f"\nFinal Bill: Rs {cart_total:.2f}")
```

```
Enter item name:  MILK
Enter quantity:  2
Not enough stock or item not found!
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship (≥80% weighted).

[18]:
```
weights = (0.10,0,5,0.15,0.5,0.10)
marks = []
for i in range(5):
    m = float(input(f"Enter marks for subject {i+1}: "))
    marks.append(m)
weighted_avg = sum(m * w for m, w in zip(marks, weights))
if weighted_avg >= 90:
    grade = "A"
elif weighted_avg >= 75:
    grade = "B"
elif weighted_avg >= 60:
    grade = "C"
elif weighted_avg >= 40:
    grade = "D"
else:
    grade = "Fail"
scholarship = "Yes" if weighted_avg >= 80 else "No"
print(f"\nWeighted Average: {weighted_avg:.2f}")
print(f"Grade: {grade}")
print(f"Scholarship Eligible: {scholarship}")
```

```
Enter marks for subject 1:  25
Enter marks for subject 2:  20
Enter marks for subject 3:  22
Enter marks for subject 4:  23
Enter marks for subject 5:  24

Weighted Average: 127.95
Grade: A
Scholarship Eligible: Yes
```

Q4. BMI + Health Recommendation

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

[37]:
```
weight = float(input("Enter weight (kg): "))
height = float(input("Enter height (m): "))
age = int(input("Enter age: "))
bmi = weight / (height ** 2)
print(f"\nYour BMI: {bmi:.2f}")
if bmi < 18.5:
    print("Status: Underweight")
    print("Suggestion: Eat more protein-rich food, follow a strength exercise plan.")
elif 18.5 <= bmi < 25:
    print("Status: Normal")
    print("Suggestion: Maintain balanced diet and regular activity.")
elif 25 <= bmi < 30:
    print("Status: Overweight")
    print("Suggestion: Reduce sugary food, try daily walking/jogging.")
else:
    print("Status: Obese")
    print("Suggestion: Follow low-calorie diet and consult a doctor for weight management.")
if age < 18:
    print("Note: You are still growing, focus on healthy eating and play.")
elif age > 50:
    print("Note: Pay attention to joint health and do light exercises.")
```

```
Enter weight (kg):  50
Enter height (m):  1.5
Enter age:  13

Your BMI: 22.22
Status: Normal
Suggestion: Maintain balanced diet and regular activity.
Note: You are still growing, focus on healthy eating and play.
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

[41]:
```
rates = {"call": 2, "sms": 1, "data": 5}
balance = float(input("Enter initial balance: "))
initial = balance
choice = input("Enter usage type (call/sms/data): ").lower()
amount = float(input(f"Enter {choice} usage: "))
cost = rates[choice] * amount
if cost <= balance:
    balance -= cost
    print(f"Used {amount} {choice}, cost = {cost}, Remaining balance = {balance}")
else:
    print("Not enough balance!")
if balance < 0.2 * initial:
    print("⚠ Please recharge soon!")
recharge = float(input("Enter recharge amount: "))
if recharge > 1000:
    recharge += 0.1 * recharge
balance += recharge
print(f"Final Balance = {balance}")
```

```
Enter initial balance:  25
Enter usage type (call/sms/data):  call
Enter call usage:  5
Used 5.0 call, cost = 10.0, Remaining balance = 15.0
Enter recharge amount:  100
Final Balance = 115.0
```

LAB-02

TASKS:

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

```
]: bill = float(input("Enter bill amount: "))
   member = input("Enter membership type (Regular/Gold/Platinum): ").lower()
   disc = 0
   if member == "gold":
       disc = 0.10
   elif member == "platinum":
       disc = 0.15
   discount_amt = bill * disc
   bill_after_disc = bill - discount_amt
   extra_disc = 0
   if bill > 10000:
       extra_disc = bill_after_disc * 0.05
   bill_after_disc -= extra_disc
   tax = bill_after_disc * 0.10
   final_amt = bill_after_disc + tax
   print("\n--- Receipt ---")
   print(f"Original Bill   : {bill}")
   print(f"Membership Disc : -{discount_amt:.2f}")
   print(f"Extra Discount  : -{extra_disc:.2f}")
   print(f"Subtotal        : {bill_after_disc:.2f}")
   print(f"Tax (10%)       : +{tax:.2f}")
   print(f"Final Amount    : {final_amt:.2f}")
   Enter bill amount:  500
   Enter membership type (Regular/Gold/Platinum):  Regular

   --- Receipt ---
   Original Bill   : 500.0
   Membership Disc : -0.00
   Extra Discount  : -0.00
   Subtotal        : 500.00
   Tax (10%)       : +50.00
   Final Amount    : 550.00
```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest "Admit in ICU."

If urgent → suggest "Immediate doctor attention."

Use nested conditions.

```
5]: temp = float(input("Enter temperature (°C): "))
    pulse = int(input("Enter pulse (bpm): "))
    oxy = int(input("Enter oxygen level (%): "))
    bp = int(input("Enter blood pressure (mmHg): "))

    if temp > 40 or oxy < 85 or bp > 180 or pulse > 150:
        print("Category: Critical")
        print("Suggestion: Admit in ICU.")
    else:
        if temp > 38 or oxy < 90 or bp > 150 or pulse > 120:
            print("Category: Urgent")
            print("Suggestion: Immediate doctor attention.")
        else:
            if temp > 37.5 or oxy < 95 or bp > 130 or pulse > 100:
                print("Category: Observation")
            else:
                print("Category: Stable")
```

```
Enter temperature (°C):  23
Enter pulse (bpm):  92
Enter oxygen level (%):  180
Enter blood pressure (mmHg):  90
Category: Stable
```

Q8. Online Examination Grader with Negative Marking

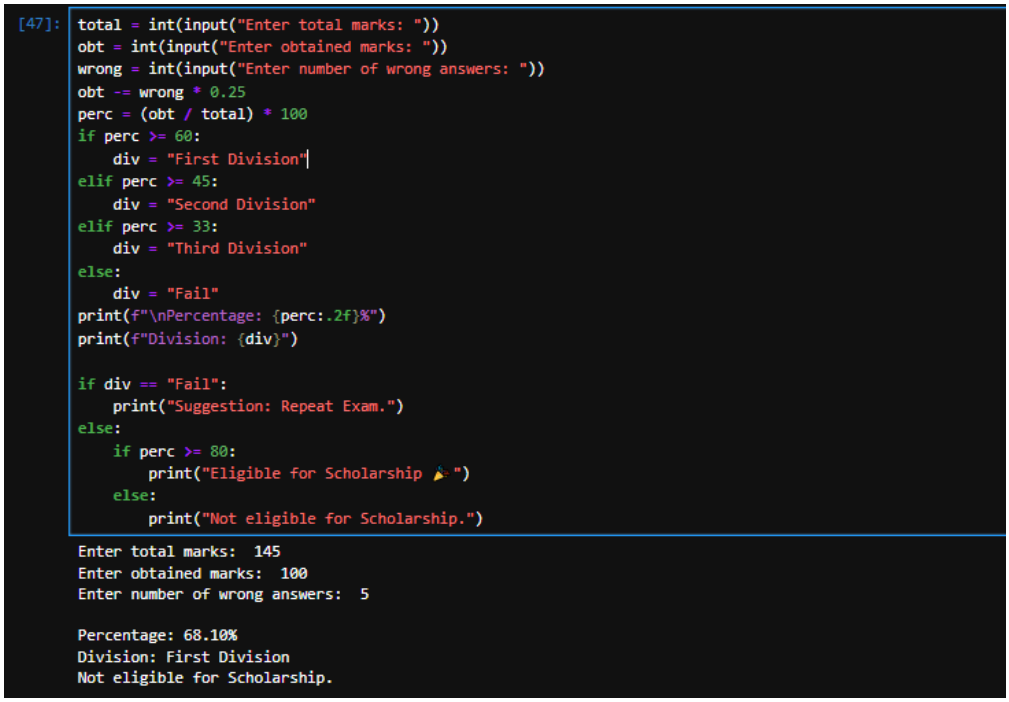
Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."



```
[47]: total = int(input("Enter total marks: "))
      obt = int(input("Enter obtained marks: "))
      wrong = int(input("Enter number of wrong answers: "))
      obt -= wrong * 0.25
      perc = (obt / total) * 100
      if perc >= 60:
          div = "First Division"
      elif perc >= 45:
          div = "Second Division"
      elif perc >= 33:
          div = "Third Division"
      else:
          div = "Fail"
      print(f"\nPercentage: {perc:.2f}%")
      print(f"Division: {div}")

      if div == "Fail":
          print("Suggestion: Repeat Exam.")
      else:
          if perc >= 80:
              print("Eligible for Scholarship 🎉")
          else:
              print("Not eligible for Scholarship.")

      Enter total marks:  145
      Enter obtained marks:  100
      Enter number of wrong answers:  5

      Percentage: 68.10%
      Division: First Division
      Not eligible for Scholarship.
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

```
[49]: prices = {"normal": 500, "3d": 700, "imax": 1000}
      age = int(input("Enter age: "))
      showtime = int(input("Enter showtime (24hr format, e.g. 21 for 9 PM): "))
      t_type = input("Enter ticket type (Normal/3D/IMAX): ").lower()
      if age < 12 and showtime >= 22:
          print("Not allowed: Children under 12 cannot attend late-night shows.")
      else:
          price = prices[t_type]
          if age >= 60:
              price *= 0.70
          if 18 <= showtime <= 22:
              price *= 1.10
          print(f"Final Ticket Price: Rs {price:.0f}")

      Enter age:  30
      Enter showtime (24hr format, e.g. 21 for 9 PM):  21
      Enter ticket type (Normal/3D/IMAX):  3D
      Final Ticket Price: Rs 770
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend "light formal suit."

```
[53]: temp = int(input("Enter temperature (°C): "))
      weather = input("Enter weather (Rainy/Sunny/Cold): ").lower()
      event = input("Enter event type (Casual/Business/Party): ").lower()
      if event == "business" and temp > 35:
          outfit = "Light formal suit"
      elif event == "casual":
          outfit = "T-shirt and jeans"
      elif event == "party":
          outfit = "Stylish dress or suit"
      else:
          outfit = "Formal outfit"
      acc = []
      if weather == "rainy":
          acc.append("Umbrella")
      if weather == "cold":
          acc.append("Gloves")
      if weather == "sunny":
          acc.append("Sunglasses")
      print("\n--- Outfit Recommendation ---")
      print(f"Outfit: {outfit}")
      if acc:
          print("Accessories: " + ", ".join(acc))
      else:
          print("Accessories: None")
```

```
Enter temperature (°C):  32
Enter weather (Rainy/Sunny/Cold):  Sunny
Enter event type (Casual/Business/Party):  Party

--- Outfit Recommendation ---
Outfit: Stylish dress or suit
Accessories: Sunglasses
```

LAB-03

TASKS:

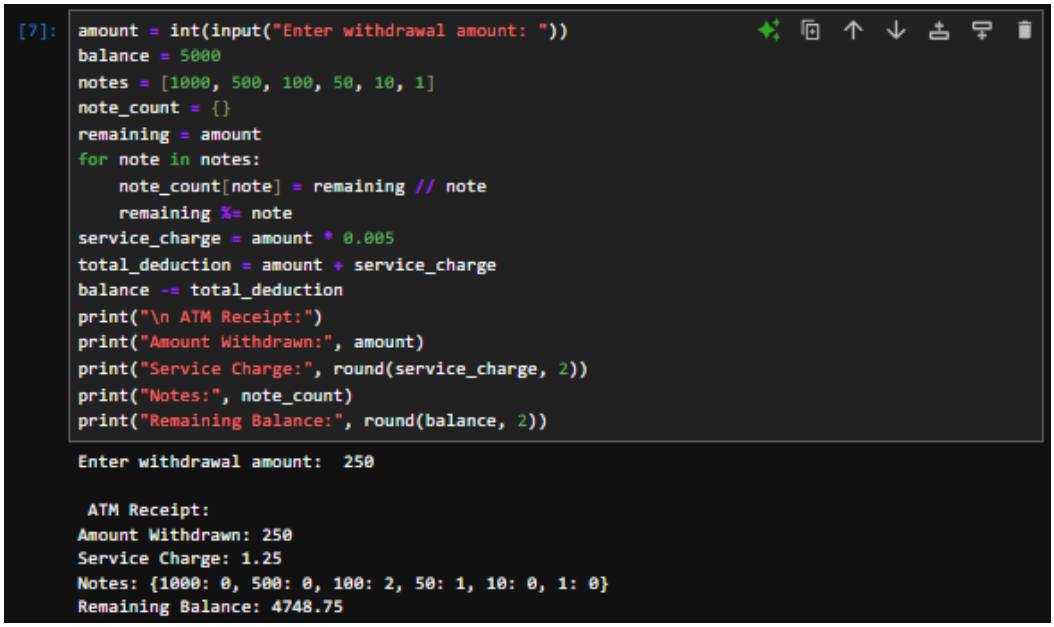
Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.



```
[7]: amount = int(input("Enter withdrawal amount: "))
     balance = 5000
     notes = [1000, 500, 100, 50, 10, 1]
     note_count = {}
     remaining = amount
     for note in notes:
         note_count[note] = remaining // note
         remaining %= note
     service_charge = amount * 0.005
     total_deduction = amount + service_charge
     balance -= total_deduction
     print("\n ATM Receipt:")
     print("Amount Withdrawn:", amount)
     print("Service Charge:", round(service_charge, 2))
     print("Notes:", note_count)
     print("Remaining Balance:", round(balance, 2))

     Enter withdrawal amount:  250

      ATM Receipt:
     Amount Withdrawn: 250
     Service Charge: 1.25
     Notes: {1000: 0, 500: 0, 100: 2, 50: 1, 10: 0, 1: 0}
     Remaining Balance: 4748.75
```

Q12. Library Fine Calculator with Membership Levels

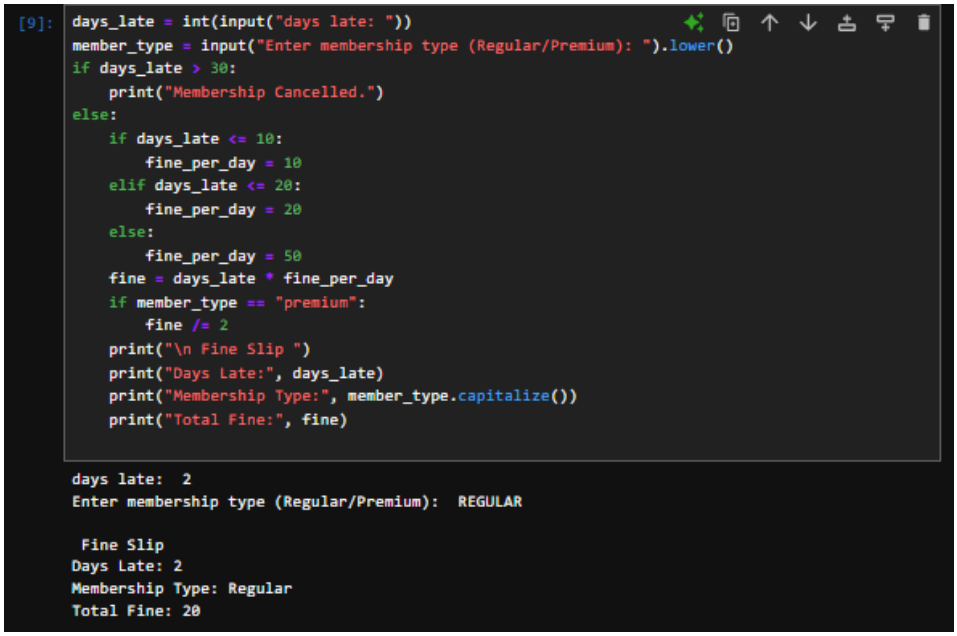
Input days late.

Apply fines (10, 20, 50/day).

If days > 30 → "Membership Cancelled."

If user is Premium member, reduce fine by 50%.

Print detailed fine slip.



```
[9]: days_late = int(input("days late: "))
member_type = input("Enter membership type (Regular/Premium): ").lower()
if days_late > 30:
    print("Membership Cancelled.")
else:
    if days_late <= 10:
        fine_per_day = 10
    elif days_late <= 20:
        fine_per_day = 20
    else:
        fine_per_day = 50
    fine = days_late * fine_per_day
    if member_type == "premium":
        fine /= 2
    print("\n Fine Slip ")
    print("Days Late:", days_late)
    print("Membership Type:", member_type.capitalize())
    print("Total Fine:", fine)

days late:  2
Enter membership type (Regular/Premium):  REGULAR

 Fine Slip
Days Late: 2
Membership Type: Regular
Total Fine: 20
```

Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).

Print max, min, average.

Print months above average.

Use loop + condition to print “Growth” or “Decline” compared to last month.

```
[11]: sales = [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
max_sale = max(sales)
min_sale = min(sales)
avg_sale = sum(sales) / len(sales)
print("Max:", max_sale)
print("Min:", min_sale)
print("AvG:", avg_sale)
print("\nMonths above average:")
for i, s in enumerate(sales, 1):
    if s > avg_sale:
        print("Month", i, ":", s)
print("\nGrowth/Decline:")
for i in range(1, len(sales)):
    if sales[i] > sales[i-1]:
        print("Month", i+1, "Growth")
    else:
        print("Month", i+1, "Decline")
```

```
Max: 21
Min: 10
AvG: 15.5

Months above average:
Month 7 : 16
Month 8 : 17
Month 9 : 18
Month 10 : 19
Month 11 : 20
Month 12 : 21

Growth/Decline:
Month 2 Growth
Month 3 Growth
Month 4 Growth
Month 5 Growth
Month 6 Growth
Month 7 Growth
Month 8 Growth
Month 9 Growth
Month 10 Growth
Month 11 Growth
Month 12 Growth
```

Q14. Multiplication Puzzle with Random Blanks

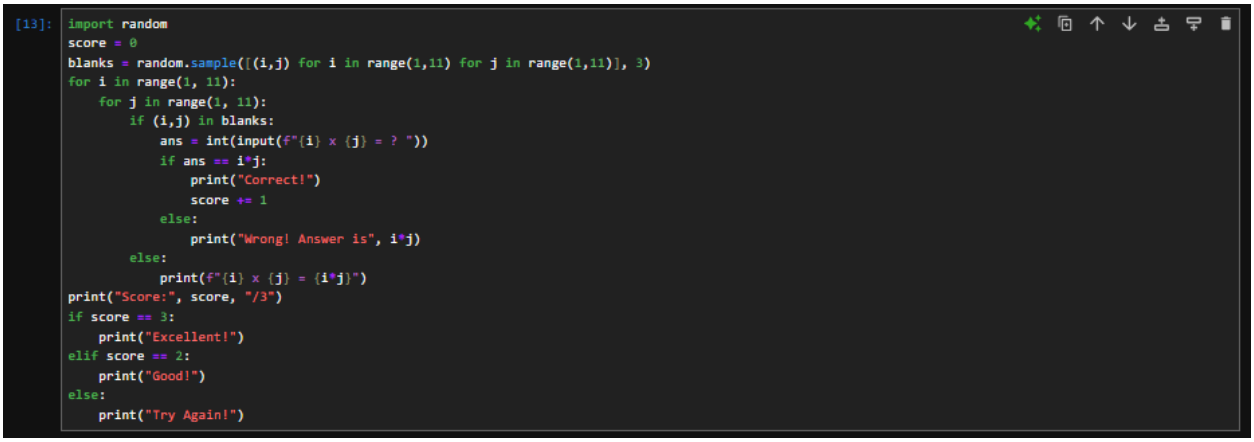
Generate a multiplication table (1–10).

Randomly replace 3 results with "??".

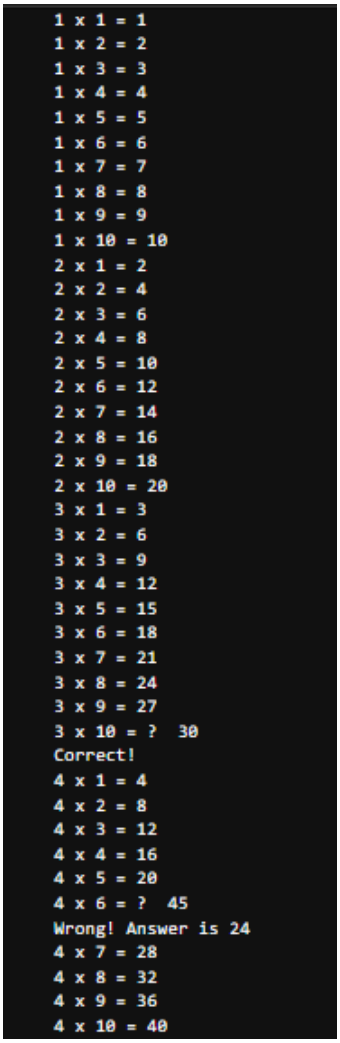
Ask user to guess values.

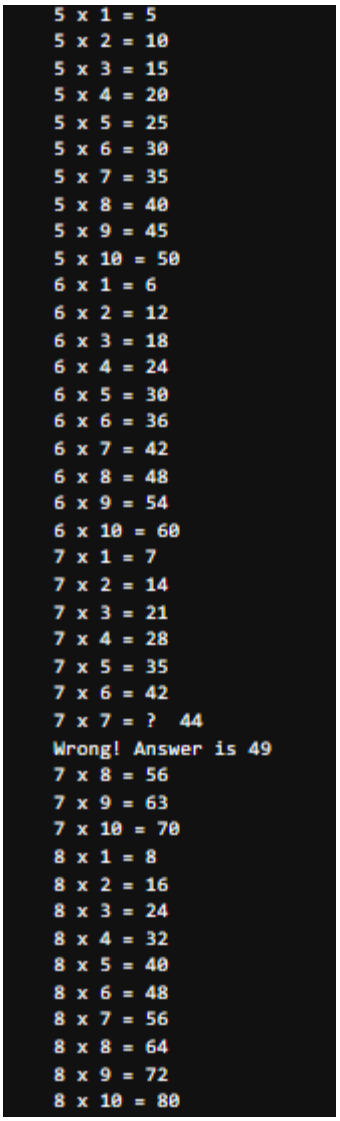
Track score.

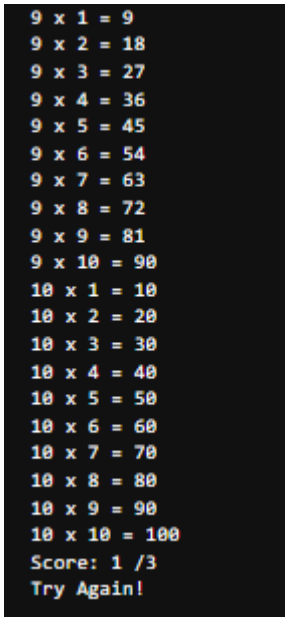
Give performance remark (Excellent/Good/Try Again).



```
[13]: import random
score = 0
blanks = random.sample([(i,j) for i in range(1,11) for j in range(1,11)], 3)
for i in range(1, 11):
    for j in range(1, 11):
        if (i,j) in blanks:
            ans = int(input(f"{i} x {j} = ? "))
            if ans == i*j:
                print("Correct!")
                score += 1
            else:
                print("Wrong! Answer is", i*j)
        else:
            print(f"{i} x {j} = {i*j}")
print("Score:", score, "/3")
if score == 3:
    print("Excellent!")
elif score == 2:
    print("Good!")
else:
    print("Try Again!")
```


1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = ? 30
Correct!
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = ? 45
Wrong! Answer is 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40


5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = ? 44
Wrong! Answer is 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80


9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
10 x 6 = 60
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
Score: 1 /3
Try Again!

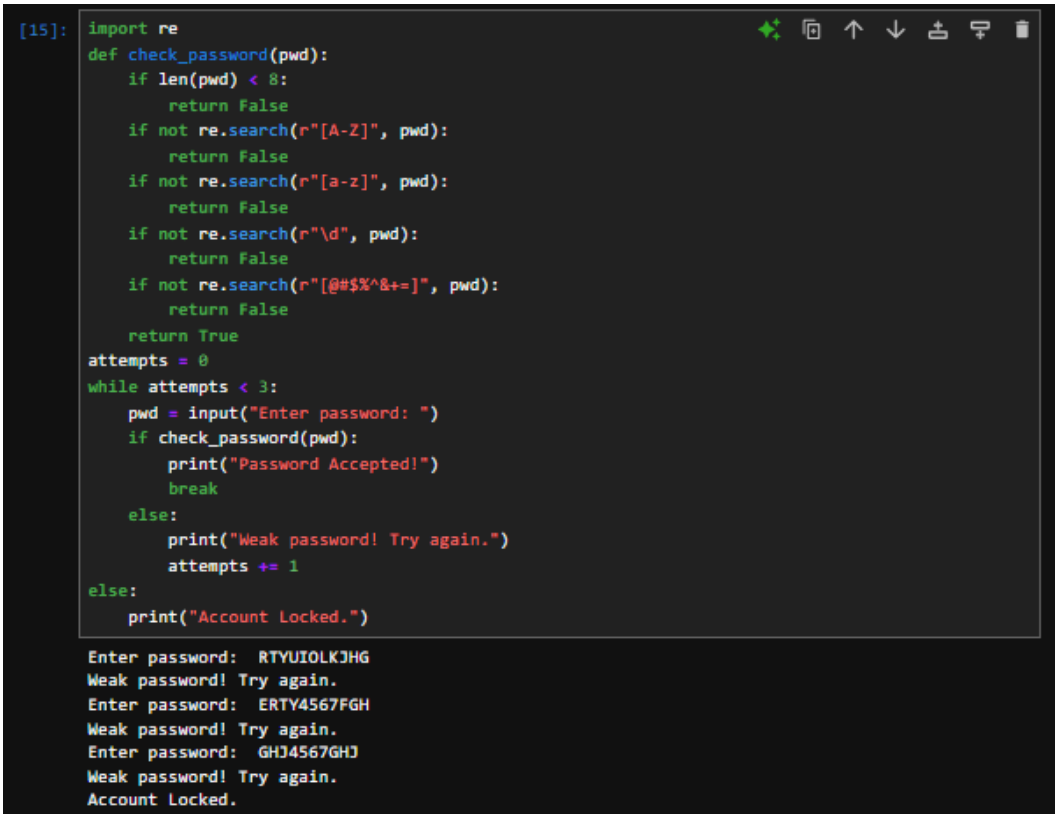
Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).

If weak → ask user to re-enter.

Allow max 3 attempts.

If still weak → "Account Locked."



```
[15]: import re
def check_password(pwd):
    if len(pwd) < 8:
        return False
    if not re.search(r"[A-Z]", pwd):
        return False
    if not re.search(r"[a-z]", pwd):
        return False
    if not re.search(r"\d", pwd):
        return False
    if not re.search(r"[@#$%^&+=]", pwd):
        return False
    return True
attempts = 0
while attempts < 3:
    pwd = input("Enter password: ")
    if check_password(pwd):
        print("Password Accepted!")
        break
    else:
        print("Weak password! Try again.")
        attempts += 1
else:
    print("Account Locked.")
```

```
Enter password:  RTYUIOLKJHG
Weak password! Try again.
Enter password:  ERTY4567FGH
Weak password! Try again.
Enter password:  GHJ4567GHJ
Weak password! Try again.
Account Locked.
```

LAB-04

TASKS:

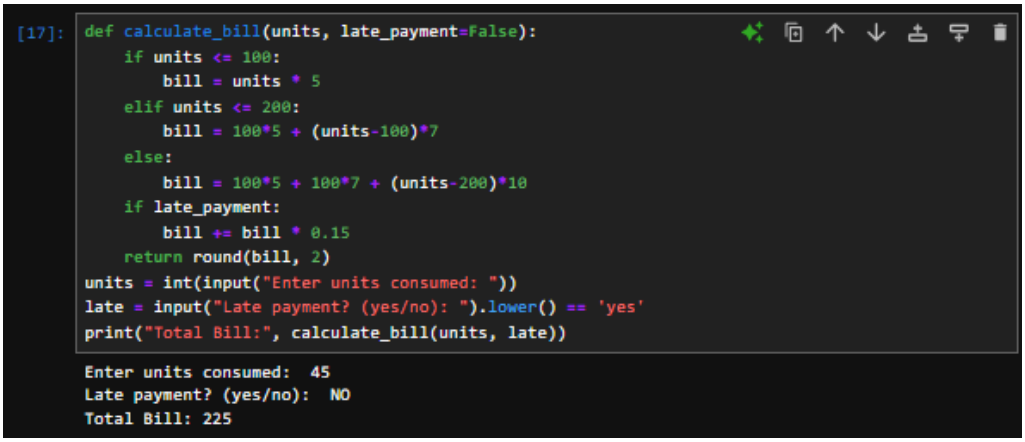
Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):

Apply slab rates (5/7/10 Rs).

If late_payment → add 15% penalty.

Return bill breakdown.



```
[17]: def calculate_bill(units, late_payment=False):
          if units <= 100:
              bill = units * 5
          elif units <= 200:
              bill = 100*5 + (units-100)*7
          else:
              bill = 100*5 + 100*7 + (units-200)*10
          if late_payment:
              bill += bill * 0.15
          return round(bill, 2)
      units = int(input("Enter units consumed: "))
      late = input("Late payment? (yes/no): ").lower() == 'yes'
      print("Total Bill:", calculate_bill(units, late))

      Enter units consumed:  45
      Late payment? (yes/no):  NO
      Total Bill: 225
```

Q17. Cricket Match Predictor with Multiple Conditions

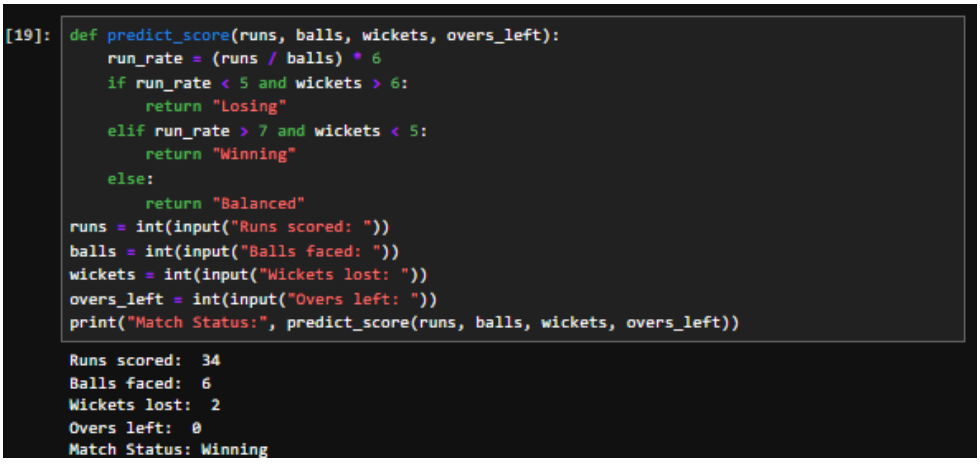
Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → "Losing."

If run rate > 7 and wickets < 5 → "Winning."

Else → "Balanced."



```
[19]: def predict_score(runs, balls, wickets, overs_left):
          run_rate = (runs / balls) * 6
          if run_rate < 5 and wickets > 6:
              return "Losing"
          elif run_rate > 7 and wickets < 5:
              return "Winning"
          else:
              return "Balanced"
      runs = int(input("Runs scored: "))
      balls = int(input("Balls faced: "))
      wickets = int(input("Wickets lost: "))
      overs_left = int(input("Overs left: "))
      print("Match Status:", predict_score(runs, balls, wickets, overs_left))

      Runs scored:  34
      Balls faced:  6
      Wickets lost:  2
      Overs left:  0
      Match Status: Winning
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):

Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.

If days > 7 → 20% discount.

If member → extra 10% discount.

Add 12% tax.

Return bill slip.

```
[23]: def book_room(type, days, is_member=False):
          prices = {"Standard":2000, "Deluxe":4000, "Suite":6000}
          cost = prices[type]*days
          if days > 7:
              cost *= 0.8
          if is_member:
              cost *= 0.9
          tax = cost * 0.12
          total = cost + tax
          return round(total, 2)
      room = input("Room type (Standard/Deluxe/Suite): ")
      days = int(input("Number of days: "))
      member = input("Are you a member? (yes/no): ").lower() == 'yes'
      print("Total Bill:", book_room(room, days, member))

      Room type (Standard/Deluxe/Suite):  Suite
      Number of days:  4
      Are you a member? (yes/no):  yes
      Total Bill: 24192.0
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):

Economy: 10/unit, Business: 25/unit, First: 40/unit.

Add fuel surcharge 500.

If international → add 18% tax.

If booking in advance (>30 days) → 10% discount.

```
def calculate_fare(distance, class_type, international=False, advance_days=0):
    rates = {"Economy":10, "Business":25, "First":40}
    fare = distance * rates[class_type] + 500
    if international:
        fare *= 1.18
    if advance_days > 30:
        fare *= 0.9
    return round(fare, 2)
distance = int(input("Distance: "))
cls = input("Class (Economy/Business/First): ")
intl = input("International? (yes/no): ").lower() == 'yes'
days = int(input("Days in advance booked: "))
print("Total Fare:", calculate_fare(distance, cls, intl, days))

Distance:  45
Class (Economy/Business/First):  First
International? (yes/no):  no
Days in advance booked:  0
Total Fare: 2300
```

Q20. Smart Vending Machine with Wallet System

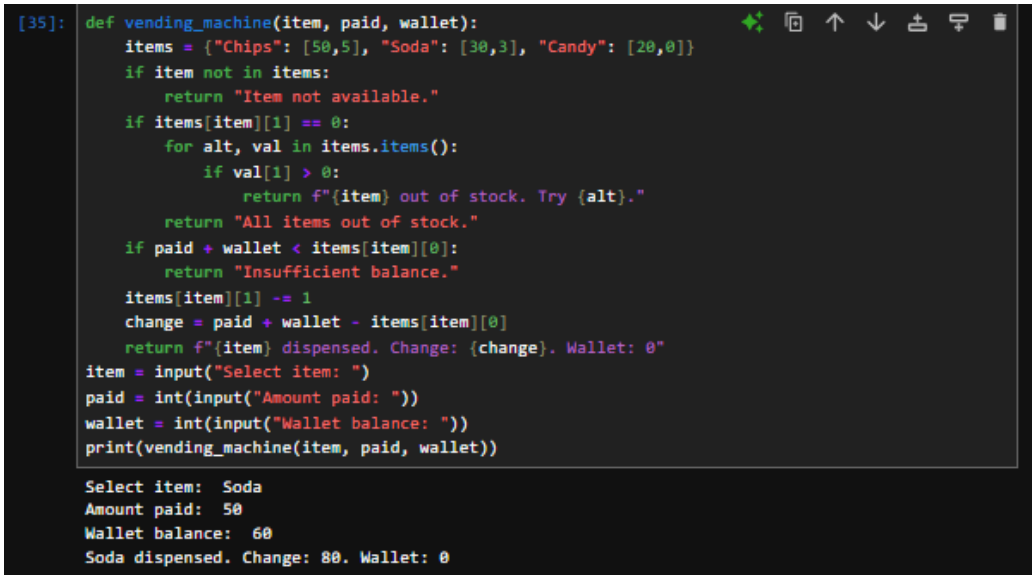
Function vending_machine(item, amount_paid, wallet_balance):

Items stored in dict with stock + price.

If sufficient → dispense + return change.

Update wallet balance.

If stock = 0 → suggest alternative item.



```
[35]: def vending_machine(item, paid, wallet):
          items = {"Chips": [50,5], "Soda": [30,3], "Candy": [20,0]}
          if item not in items:
              return "Item not available."
          if items[item][1] == 0:
              for alt, val in items.items():
                  if val[1] > 0:
                      return f"{item} out of stock. Try {alt}."
              return "All items out of stock."
          if paid + wallet < items[item][0]:
              return "Insufficient balance."
          items[item][1] -= 1
          change = paid + wallet - items[item][0]
          return f"{item} dispensed. Change: {change}. Wallet: 0"
      item = input("Select item: ")
      paid = int(input("Amount paid: "))
      wallet = int(input("Wallet balance: "))
      print(vending_machine(item, paid, wallet))

      Select item:  Soda
      Amount paid:  50
      Wallet balance:  60
      Soda dispensed. Change: 80. Wallet: 0
```

\

LAB-05

TASKS:

Q1: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

```
[5]: class BankAccount:
         def __init__(self, name, balance):
             self.name = name
             self.balance = balance
         def deposit(self, amount):
             self.balance += amount
         def withdraw(self, amount):
             if amount <= self.balance:
                 self.balance -= amount
             else:
                 print("not enough balance")
         def show(self):
             print("Balance:", self.balance)
     acc = BankAccount("eman", 5000)
     acc.deposit(500)
     acc.withdraw(100)
     acc.show()

     Balance: 5400
```

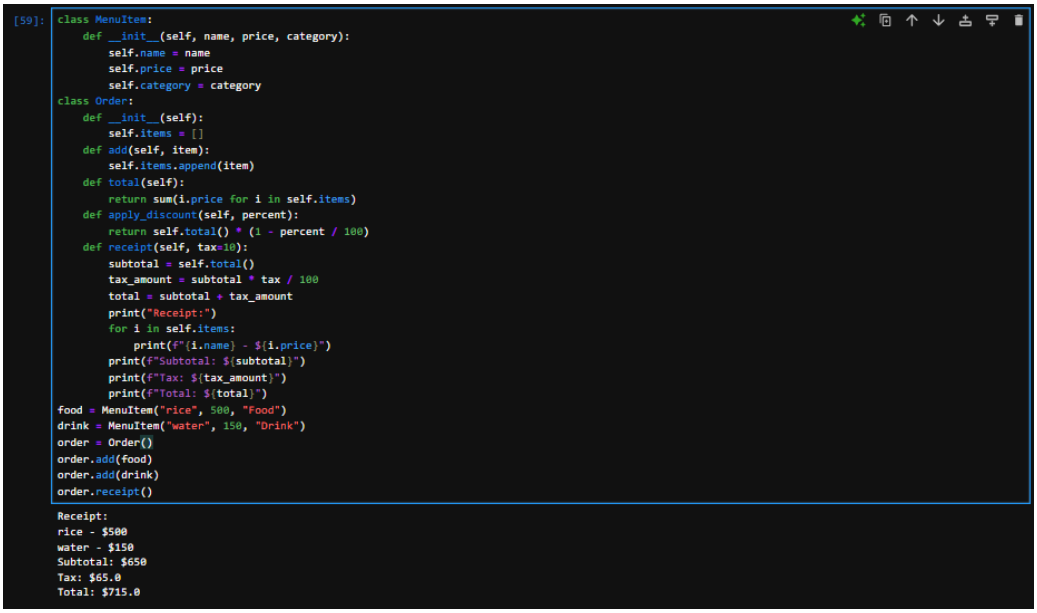
Q2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

```
[43]: class Book:
          def __init__(self, title, author):
              self.title = title
              self.author = author
              self.available = True
      class Library:
          def __init__(self):
              self.books = []
          def add(self, book):
              self.books.append(book)
          def search(self, name):
              for b in self.books:
                  if name in b.title or name in b.author:
                      print(b.title, "-", b.author, "-", "Available" if b.available else "Checked out")
          def checkout(self, title):
              for b in self.books:
                  if b.title == title and b.available:
                      b.available = False
                      print("Checked out:", b.title)
          def return_book(self, title):
              for b in self.books:
                  if b.title == title and not b.available:
                      b.available = True
                      print("Returned:", b.title)
      lib = Library()
      lib.add(Book("english", "writer1"))
      lib.add(Book("maths", "writer2"))
      lib.search("english")
      lib.checkout("english")

      english - writer1 - Available
      Checked out: english
```

Q3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

```
[51]: class Student:
          def __init__(self, id, name):
              self.id = id
              self.name = name
              self.grades = []
          def add_grade(self, grade):
              self.grades.append(grade)
          def average(self):
              return sum(self.grades) / len(self.grades)
          def letter(self):
              avg = self.average()
              if avg >= 90: return 'A'
              elif avg >= 80: return 'B'
              elif avg >= 70: return 'C'
              elif avg >= 60: return 'D'
              else: return 'F'
          def show(self):
              print(f"ID: {self.id}, Name: {self.name}")
              print("Marks:", self.grades)
              print("Average:", self.average())
              print("Grade:", self.letter())
      s = Student(5, "ayesha")
      s.add_grade(70)
      s.add_grade(80)
      s.add_grade(65)
      s.show()

      ID: 5, Name: ayesha
      Marks: [70, 80, 65]
      Average: 71.66666666666667
      Grade: C
```

Q4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.



```
[59]: class MenuItem:
          def __init__(self, name, price, category):
              self.name = name
              self.price = price
              self.category = category
      class Order:
          def __init__(self):
              self.items = []
          def add(self, item):
              self.items.append(item)
          def total(self):
              return sum(i.price for i in self.items)
          def apply_discount(self, percent):
              return self.total() * (1 - percent / 100)
          def receipt(self, tax=10):
              subtotal = self.total()
              tax_amount = subtotal * tax / 100
              total = subtotal + tax_amount
              print("Receipt:")
              for i in self.items:
                  print(f"{i.name} - ${i.price}")
              print(f"Subtotal: ${subtotal}")
              print(f"Tax: ${tax_amount}")
              print(f"Total: ${total}")
      food = MenuItem("rice", 500, "Food")
      drink = MenuItem("water", 150, "Drink")
      order = Order()
      order.add(food)
      order.add(drink)
      order.receipt()

      Receipt:
      rice - $500
      water - $150
      Subtotal: $650
      Tax: $65.0
      Total: $715.0
```

Q5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

[65]:
```
class Vehicle:
    def __init__(self, plate, model, price):
        self.plate = plate
        self.model = model
        self.price = price
        self.available = True
    def rent(self, days):
        if self.available:
            self.available = False
            return self.price * days
        return 0
    def return_vehicle(self):
        self.available = True
class Car(Vehicle): pass
class Motorcycle(Vehicle): pass
class Truck(Vehicle): pass
c = Car("car1", "honda", 50000)
print("Cost:", c.rent(5))
c.return_vehicle()
```

```
Cost: 250000
```

LAB-06

TASKS:

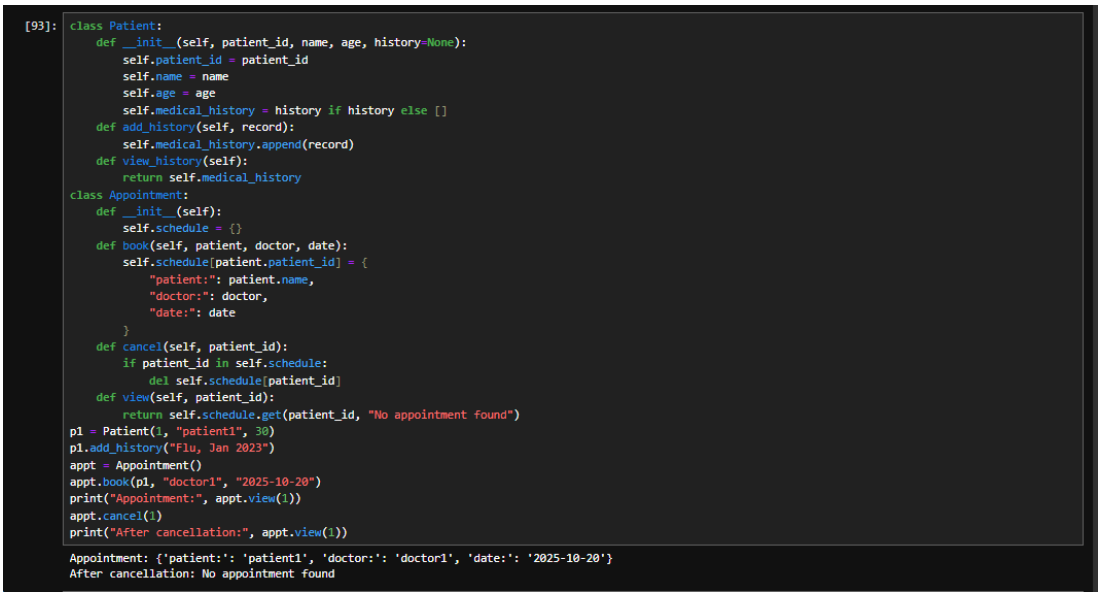
Q6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

```
[71]: from datetime import date
      class Employee:
          def __init__(self, emp_id, name, position, salary, start_date):
              self.emp_id = emp_id
              self.name = name
              self.position = position
              self.salary = salary
              self.start_date = start_date
          def monthly_salary(self):
              return self.salary / 12
          def annual_salary(self):
              return self.salary
          def apply_raise(self, percent):
              self.salary += self.salary * (percent / 100)
          def employment_duration(self):
              today = date.today()
              duration = today - self.start_date
              return duration.days // 365
          def details(self):
              return f"ID: {self.emp_id}, Name: {self.name}, Position: {self.position}, Salary: ${self.salary:.2f}"
      emp = Employee(101, "eman", "manager", 60000, date(2023,7,1))
      print(emp.details())
      print("Monthly Salary:", emp.monthly_salary())
      print("Employment Duration:", emp.employment_duration())
      emp.apply_raise(10)
      print("New Salary:", emp.annual_salary())

      ID: 101, Name: eman, Position: manager, Salary: $60000.00
      Monthly Salary: 5000.0
      Employment Duration: 2
      New Salary: 66000.0
```

Q7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

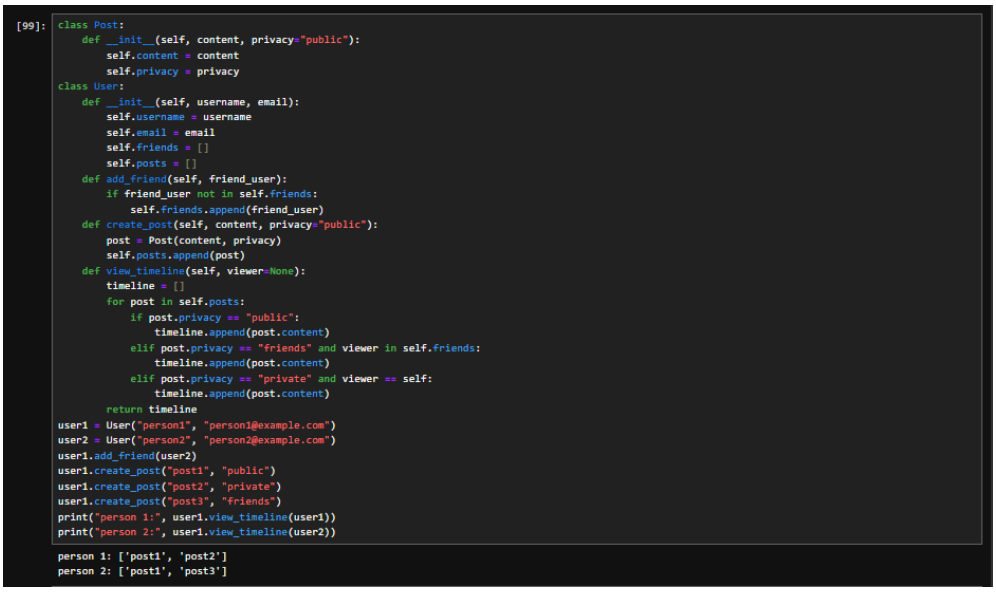
```
[87]: class Product:
          def __init__(self, product_id, name, price, stock):
              self.product_id = product_id
              self.name = name
              self.price = price
              self.stock = stock
          def update_stock(self, quantity):
              self.stock -= quantity
      class ShoppingCart:
          def __init__(self):
              self.items = {}
          def add_product(self, product, quantity):
              if product.stock >= quantity:
                  self.items[product] = self.items.get(product, 0) + quantity
              else:
                  print(f"Not enough stock for {product.name}")
          def remove_product(self, product):
              if product in self.items:
                  del self.items[product]
          def total_cost(self):
              return sum(product.price * qty for product, qty in self.items.items())
          def checkout(self):
              for product, qty in self.items.items():
                  product.update_stock(qty)
              self.items.clear()
              print("Checkout complete. Inventory updated.")
      p1 = Product(1, "rice", 5000, 5)
      p2 = Product(2, "flour", 4500, 5)
      cart = ShoppingCart()
      cart.add_product(p1, 1)
      cart.add_product(p2, 1)
      print("Total Cost:", cart.total_cost())
      cart.checkout()
      print("Remaining stock:", p1.stock, p2.stock)

      Total Cost: 9500
      Checkout complete. Inventory updated.
      Remaining stock: 4 4
```

Q8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.



```
[93]: class Patient:
          def __init__(self, patient_id, name, age, history=None):
              self.patient_id = patient_id
              self.name = name
              self.age = age
              self.medical_history = history if history else []
          def add_history(self, record):
              self.medical_history.append(record)
          def view_history(self):
              return self.medical_history
      class Appointment:
          def __init__(self):
              self.schedule = {}
          def book(self, patient, doctor, date):
              self.schedule[patient.patient_id] = {
                  "patient:": patient.name,
                  "doctor:": doctor,
                  "date:": date
              }
          def cancel(self, patient_id):
              if patient_id in self.schedule:
                  del self.schedule[patient_id]
          def view(self, patient_id):
              return self.schedule.get(patient_id, "No appointment found")
      p1 = Patient(1, "patient1", 30)
      p1.add_history("Flu, Jan 2023")
      appt = Appointment()
      appt.book(p1, "doctor1", "2025-10-20")
      print("Appointment:", appt.view(1))
      appt.cancel(1)
      print("After cancellation:", appt.view(1))

      Appointment: {'patient:': 'patient1', 'doctor:': 'doctor1', 'date:': '2025-10-20'}
      After cancellation: No appointment found
```

Q9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.



```
[99]: class Post:
          def __init__(self, content, privacy="public"):
              self.content = content
              self.privacy = privacy
      class User:
          def __init__(self, username, email):
              self.username = username
              self.email = email
              self.friends = []
              self.posts = []
          def add_friend(self, friend_user):
              if friend_user not in self.friends:
                  self.friends.append(friend_user)
          def create_post(self, content, privacy="public"):
              post = Post(content, privacy)
              self.posts.append(post)
          def view_timeline(self, viewer=None):
              timeline = []
              for post in self.posts:
                  if post.privacy == "public":
                      timeline.append(post.content)
                  elif post.privacy == "friends" and viewer in self.friends:
                      timeline.append(post.content)
                  elif post.privacy == "private" and viewer == self:
                      timeline.append(post.content)
              return timeline
      user1 = User("person1", "person1@example.com")
      user2 = User("person2", "person2@example.com")
      user1.add_friend(user2)
      user1.create_post("post1", "public")
      user1.create_post("post2", "private")
      user1.create_post("post3", "friends")
      print("person 1:", user1.view_timeline(user1))
      print("person 2:", user1.view_timeline(user2))

      person 1: ['post1', 'post2']
      person 2: ['post1', 'post3']
```

Q10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

[103]:
```
from datetime import date
class Room:
    def __init__(self, room_number, room_type, price, available=True):
        self.room_number = room_number
        self.room_type = room_type
        self.price = price
        self.available = available
    def mark_unavailable(self):
        self.available = False
    def mark_available(self):
        self.available = True
class Booking:
    def __init__(self, room, check_in, check_out):
        self.room = room
        self.check_in = check_in
        self.check_out = check_out
    def total_cost(self):
        nights = (self.check_out - self.check_in).days
        return nights * self.room.price
    def confirm_booking(self):
        if self.room.available:
            self.room.mark_unavailable()
            return f"Room {self.room.room_number} booked from {self.check_in} to {self.check_out}"
        else:
            return "Room not available"
room1 = Room(101, "double", 120)
booking1 = Booking(room1, date(2025, 10, 20), date(2025, 10, 23))
print(booking1.confirm_booking())
print("Total Cost:", booking1.total_cost())
print("Room Available:", room1.available)
```

```
Room 101 booked from 2025-10-20 to 2025-10-23
Total Cost: 360
Room Available: False
```

LAB-07

TASKS:

Q11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

```
[3]: class Course:
         def __init__(self, code, name, credits):
             self.code = code
             self.name = name
             self.credits = credits
     class Student:
         def __init__(self, name, max_credits=18):
             self.name = name
             self.courses = []
             self.max_credits = max_credits
         def register(self, course):
             total = sum(c.credits for c in self.courses)
             if total + course.credits > self.max_credits:
                 print("Cannot register, credit limit exceeded.")
             elif course in self.courses:
                 print("Already registered for this course.")
             else:
                 self.courses.append(course)
                 print(f"Registered for {course.name}")
     c1 = Course("CS101", "Python", 4)
     c2 = Course("CS102", "Data Science", 8)
     s = Student("eman")
     s.register(c1)
     s.register(c2)

     Registered for Python
     Registered for Data Science
```

Q12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

```
[7]: class Product:
         def __init__(self, pid, name, qty, price):
             self.id = pid
             self.name = name
             self.qty = qty
             self.price = price
     class Inventory:
         def __init__(self):
             self.products = []
         def add_product(self, product):
             self.products.append(product)
         def update_quantity(self, pid, qty):
             for p in self.products:
                 if p.id == pid:
                     p.qty += qty
         def low_stock_alert(self, threshold=5):
             for p in self.products:
                 if p.qty < threshold:
                     print(f"{p.name} is low on stock ({p.qty} left)")
     inv = Inventory()
     p1 = Product(1, "juice", 3, 500)
     inv.add_product(p1)
     inv.low_stock_alert()

     juice is low on stock (3 left)
```

Q13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

```
[9]: class Movie:
         def __init__(self, title, showtimes):
             self.title = title
             self.showtimes = showtimes
     class Theater:
         def __init__(self, seats=10):
             self.seats = ["Available"] * seats

         def book_seat(self, seat_no):
             if self.seats[seat_no] == "Booked":
                 print("Seat already booked")
             else:
                 self.seats[seat_no] = "Booked"
                 print(f"Seat {seat_no} booked successfully")
     movie = Movie("movie", ["5PM", "8PM"])
     theater = Theater(5)
     theater.book_seat(2)

     Seat 2 booked successfully
```

Q14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

```
[11]: class Member:
          def __init__(self, name, type):
              self.name = name
              self.type = type
              self.attendance = []
      class Trainer:
          def __init__(self, name):
              self.name = name
      def schedule(member, trainer, day):
          member.attendance.append(day)
          print(f"Training scheduled for {member.name} with {trainer.name} on {day}")
      m = Member("member", "Premium")
      t = Trainer("trainer")
      schedule(m, t, "Monday")

      Training scheduled for member with trainer on Monday
```

Q15: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

[13]:
```
class Account:
    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance
class SavingsAccount(Account):
    def __init__(self, name, balance=0, interest=0.05):
        super().__init__(name, balance)
        self.interest = interest
class CheckingAccount(Account):
    def __init__(self, name, balance=0):
        super().__init__(name, balance)
def transfer(from_acc, to_acc, amount):
    if from_acc.balance >= amount:
        from_acc.balance -= amount
        to_acc.balance += amount
        print("Transfer successful")
    else:
        print("Insufficient funds")
s = SavingsAccount("person 1", 1000)
c = CheckingAccount("person 2", 500)
transfer(s, c, 200)
```

```
Transfer successful
```

LAB-08

TASKS:

Q16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

```
[15]: class Pizza:
          def __init__(self, size, crust, toppings=[]):
              self.size = size
              self.crust = crust
              self.toppings = toppings
          def price(self):
              base = 200 if self.size=="Small" else 300
              return base + 50*len(self.toppings)
      p = Pizza("Medium", "Thin", ["Cheese", "Olives"])
      print("Price:", p.price())

      Price: 400
```

Q17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

```
[17]: class Car:
          def __init__(self, model, year, color, price):
              self.model = model
              self.year = year
              self.color = color
              self.price = price
      class CarManufacturer:
          def __init__(self, name):
              self.name = name
              self.cars = []
          def produce_car(self, car):
              self.cars.append(car)
              print(f"{car.model} produced")
      m = CarManufacturer("Toyota")
      c = Car("Corolla", 2023, "Red", 20000)
      m.produce_car(c)

      Corolla produced
```

Q18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

[19]:
```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []
class Teacher:
    def __init__(self, name):
        self.name = name
class Classroom:
    def __init__(self):
        self.students = []
    def add_student(self, student):
        self.students.append(student)
    def add_grade(self, student, grade):
        student.grades.append(grade)
    def report(self):
        for s in self.students:
            avg = sum(s.grades)/len(s.grades) if s.grades else 0
            print(f"{s.name}: Average Grade = {avg}")
cls = Classroom()
s1 = Student("eman")
cls.add_student(s1)
cls.add_grade(s1, 90)
cls.report()
```

```
eman: Average Grade = 90.0
```

Q19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

[21]:
```
class Flight:
    def __init__(self, number, destination, capacity):
        self.number = number
        self.destination = destination
        self.capacity = capacity
        self.passengers = []
    def book(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            print(f"{passenger} booked")
        else:
            print("Flight full")
f = Flight("F1", "Paris", 2)
f.book("person 1")
f.book("person 2")
f.book("person 3")
```

```
person 1 booked
person 2 booked
Flight full
```

Q20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

[23]:
```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "Off"
    def turn_on(self):
        self.status = "On"
    def turn_off(self):
        self.status = "Off"
class SmartHome:
    def __init__(self):
        self.devices = []
    def add_device(self, device):
        self.devices.append(device)
    def show_status(self):
        for d in self.devices:
            print(f"{d.name}: {d.status}")
light = SmartDevice("Living Room Light")
home = SmartHome()
home.add_device(light)
light.turn_on()
home.show_status()
```

```
Living Room Light: On
```

LAB-09

TASKS:

Q1: A weather station records temperatures (°C) for 7 days:

[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- Convert the list into a NumPy array
- Find:
    - Average temperature
    - Maximum and minimum temperature
    - Temperature difference (range)

```
[24]: import numpy as np
      temp = np.array([21.1,23.4,19.8,25.0,26.5,24.1,22.0])
      print("Average:", temp.mean())
      print("Max:", temp.max())
      print("Min:", temp.min())
      print("Range:", temp.max() - temp.min())

      Average: 23.12857142857143
      Max: 26.5
      Min: 19.8
      Range: 6.699999999999999
```

Q2: A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- Create a **4×7 NumPy array**
- Find weekly totals
- Find the highest sale day (index only)

- Find the week with the most sales

```
[28]: import numpy as np
      sales = np.array([
       [12,15,14,10,9,8,7],
       [11,12,13,9,5,8,6],
       [7,9,12,10,11,13,12],
       [10,11,12,7,8,9,11]
      ])
      weekly_total = sales.sum(axis=1)
      print("Weekly totals:", weekly_total)
      print("Highest sale day index:", np.argmax(sales))
      print("Best week:", np.argmax(weekly_total))

      Weekly totals: [75 64 74 68]
      Highest sale day index: 1
      Best week: 0
```

Q3: Students received marks in a test:

[56, 78, 45, 90, 88, 76, 59, 62]

Task:

- Convert to NumPy array
- Add **5 bonus marks** to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation

```
[30]: import numpy as np
      marks = np.array([56,78,45,90,88,76,59,62])
      marks += 5
      print("Above 60:", np.sum(marks > 60))
      print("Median:", np.median(marks))
      print("Std Dev:", np.std(marks))

      Above 60: 7
      Median: 74.0
      Std Dev: 15.105876340020794
```

Q4: A smartwatch stores daily step-counts for a user for 30 days.

Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000
- Reshape into a **5×6** matrix
- Find:

- Weekly average steps
- Day with maximum steps
- Number of days user crossed 10,000 steps

```
[32]: import numpy as np
      steps = np.random.randint(2000,15000,30)
      steps = steps.reshape(5,6)
      print("Weekly avg:", steps.mean(axis=1))
      print("Max steps day:", np.argmax(steps))
      print(">10000 days:", np.sum(steps > 10000))

      Weekly avg: [ 9379.         10093.16666667  6357.83333333  9543.
        6875.66666667]
      Max steps day: 22
      >10000 days: 9
```

Q5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
  - Compute daily average
  - Identify abnormal readings > 120
- For all patients:
  - Compute overall maximum and minimum

```
[34]: import numpy as np
      hr = np.random.randint(60,150,(10,4))
      print("Daily avg per patient:", hr.mean(axis=1))
      print("Abnormal (>120):", hr > 120)
      print("Overall max:", hr.max())
      print("Overall min:", hr.min())

      Daily avg per patient: [ 91.    99.75 117.25  93.5  114.   116.   108.25 106.25 124.75  85.5 ]
      Abnormal (>120): [[False False  True False]
       [False False  True False]
       [ True  True False False]
       [False False False  True]
       [False  True  True False]
       [False False  True  True]
       [False False  True  True]
       [ True False False False]
       [ True  True  True False]
       [False False False False]]
      Overall max: 149
      Overall min: 60
```

LAB-10

TASKS:

Q6: Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- Compute difference between branches
- Calculate:
    - Average monthly sales of each branch
    - Which branch performed better overall?
- Compute **correlation** between A and B

```
[36]: import numpy as np
      hr = np.random.randint(60,150,(10,4))
      print("Daily avg per patient:", hr.mean(axis=1))
      print("Abnormal (>120):", hr > 120)
      print("Overall max:", hr.max())
      print("Overall min:", hr.min())

      Daily avg per patient: [105.25 111.5  118.5   99.   102.5  120.5   81.   114.25 102.75 103.  ]
      Abnormal (>120): [[ True False  True False]
       [False False  True False]
       [ True False  True False]
       [False False False False]
       [ True False  True False]
       [ True  True  True False]
       [False False False False]
       [False  True False  True]
       [False False  True False]
       [False False  True False]]
      Overall max: 149
      Overall min: 60
```

Q7: Given a 4×4 grayscale image:

[[100, 120, 130, 90],

[80 , 110, 140, 100],

[70 , 90 , 120, 130],

[110, 140, 150, 160]]

Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: new_pixel = pixel * 1.2
- Clip all values > 255
- Compute average brightness before & after transformation

```
[40]: import numpy as np
      img = np.array([[100,120,130,90],
                      [80,110,140,100],
                      [70,90,120,130],
                      [110,140,150,160]])
      norm = img / 255
      new_img = np.clip(img*1.2, 0, 255)
      print("Avg before:", img.mean())
      print("Avg after:", new_img.mean())

      Avg before: 115.0
      Avg after: 138.0
```

Q8: Dataset contains 100 samples with 4 features each.

Task:

- Generate a **100×4** matrix with values 1–100
- Standardize the dataset using:
- x_scaled = (x - mean) / std
- Perform:
    - Column-wise mean (should be ≈ 0)
    - Column-wise variance (should be ≈ 1)
- Verify results using NumPy functions

```
[46]: import numpy as np
      X = np.random.randint(1,101,(100,4))
      X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)
      print("Mean:", X_scaled.mean(axis=0))
      print("Variance:", X_scaled.var(axis=0))

      Mean: [-3.10862447e-17  1.99840144e-17  1.06581410e-16  4.30211422e-18]
      Variance: [1. 1. 1. 1.]
```

Q9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the **busiest sensor** (highest total density)
- Apply a threshold:
    - Replace all values above 50 with 50 (traffic cap)
- Compute energy usage:

energy = Σ(sensor_density × 0.5)

```
[44]: import numpy as np
      traffic = np.random.randint(10,80,(24,6))
      print("Hourly avg:", traffic.mean(axis=1))
      print("Busiest sensor:", np.argmax(traffic.sum(axis=0)))
      traffic = np.clip(traffic,0,50)
      energy = np.sum(traffic * 0.5)
      print("Energy usage:", energy)

      Hourly avg: [52.5        47.16666667 42.16666667 41.83333333 46.16666667 39.
       40.16666667 34.5        29.         45.16666667 55.16666667 46.33333333
       48.83333333 34.         33.16666667 56.83333333 47.16666667 44.
       24.83333333 52.16666667 43.66666667 55.33333333 54.         45.66666667]
      Busiest sensor: 4
      Energy usage: 2750.5
```

Q10: You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

F = [[5,2,1],

[3,1,4],

[6,3,2],

[4,2,5],

[7,1,3]]

Transformation matrix:

T = [[2,1,0],

[1,3,1],

[0,2,2]]

Task:

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

```
[42]: import numpy as np
      F = np.array([[5,2,1],[3,1,4],[6,3,2],[4,2,5],[7,1,3]])
      T = np.array([[2,1,0],[1,3,1],[0,2,2]])
      F_new = F @ T
      mag = np.linalg.norm(F_new, axis=1)
      print("Transformed forces:\n", F_new)
      print("Magnitudes:", mag)
      print("Highest force component:", np.argmax(mag))
      eig_val, eig_vec = np.linalg.eig(T)
      print("Eigenvalues:", eig_val)

      Transformed forces:
       [[12 13  4]
       [ 7 14  9]
       [15 19  7]
       [10 20 12]
       [15 16  7]]
      Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
      Highest force component: 3
      Eigenvalues: [4.30277564 2.         0.69722436]
```

LAB-11

SEABORN TASKS:

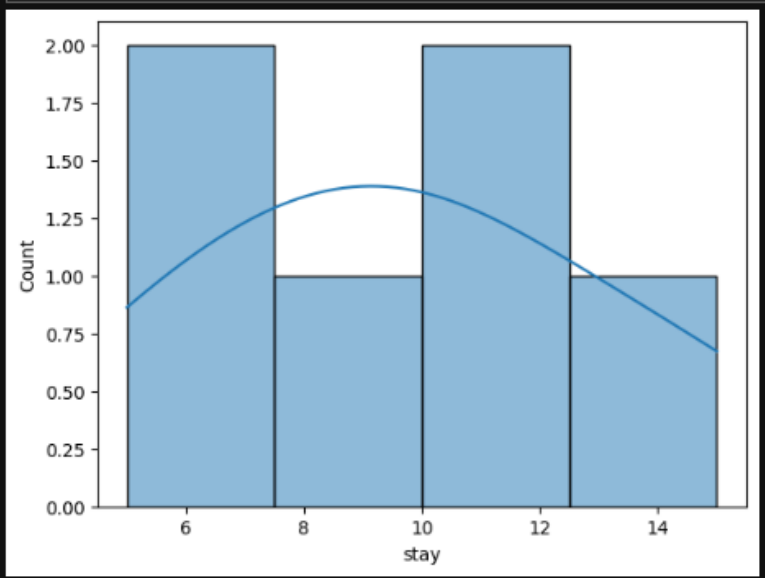
Q1. Patient Stay Analysis (Healthcare)

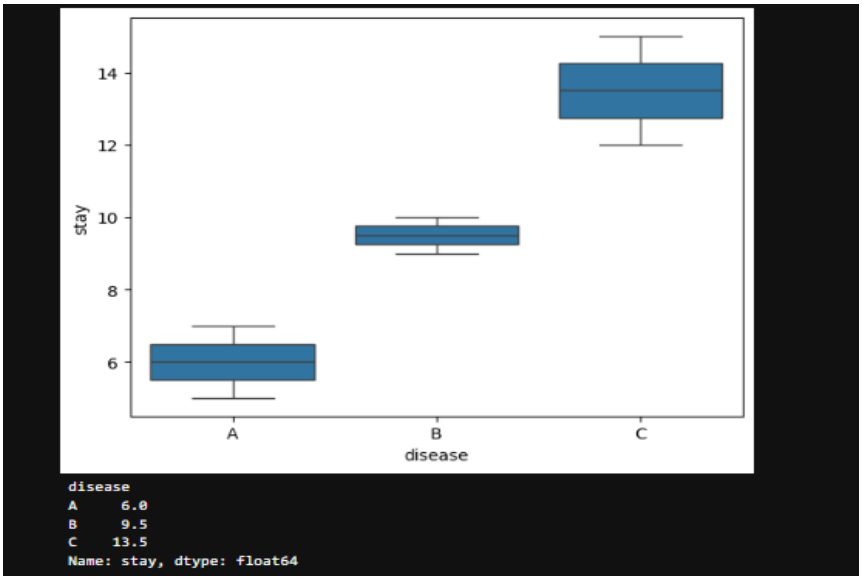
You are given a dataset containing age, disease_type, and hospital_stay_days.

Write a Python program using **Seaborn** to:

- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

```
[1]: import seaborn as sns
     import matplotlib.pyplot as plt
     import pandas as pd
     df = pd.DataFrame({
      "disease":["A","B","A","C","B","C"],
      "stay":[5,10,7,12,9,15]
     })
     sns.histplot(df["stay"], kde=True)
     plt.show()
     sns.boxplot(x="disease", y="stay", data=df)
     plt.show()
     print(df.groupby("disease")["stay"].median())
```





Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.

Write code to:

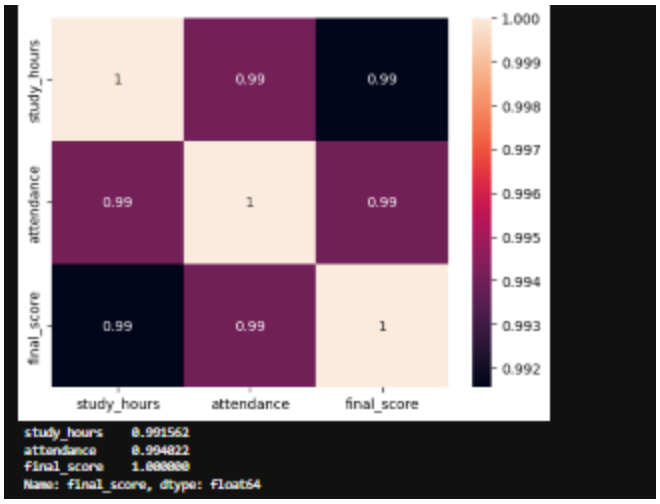
- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

Q3. Student Performance Correlation (Education)

A dataset contains study_hours, attendance, and final_score .

Using Seaborn:

- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

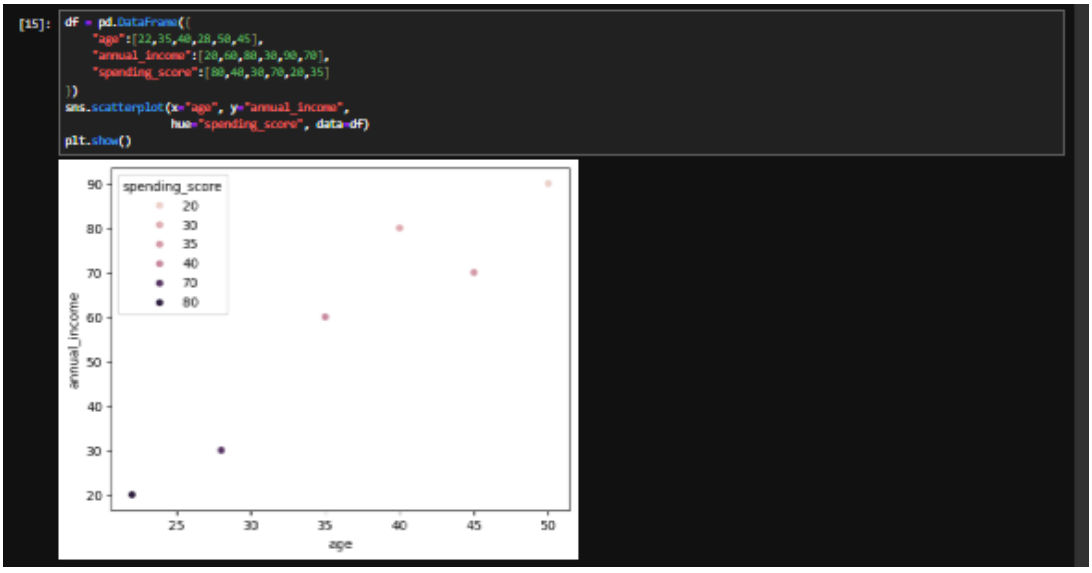


Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.

Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

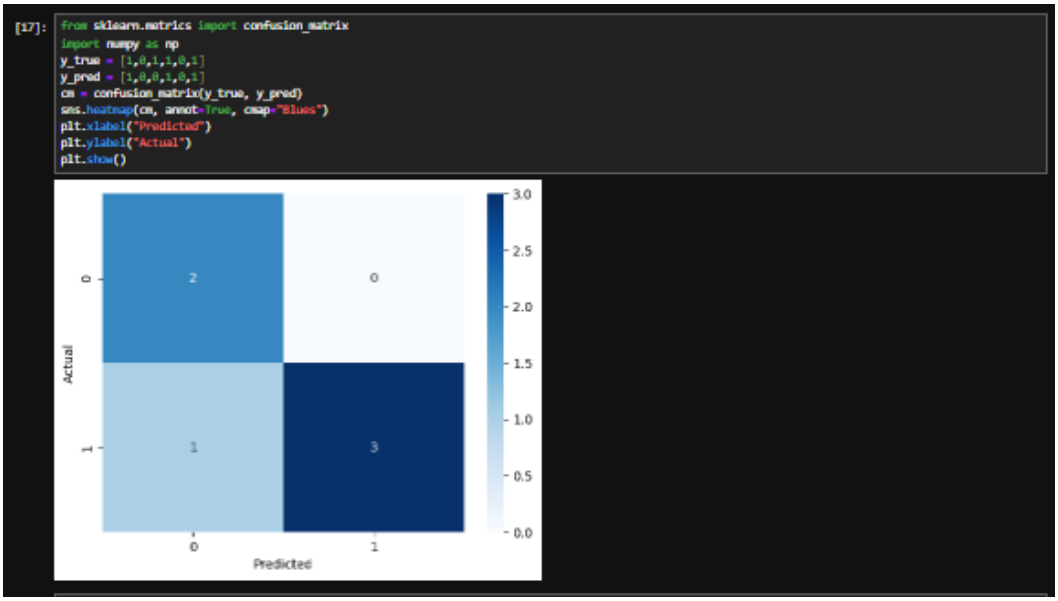


Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.

Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes



LAB-12

SCI-KIT TASKS:

Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

```
[19]: from sklearn.preprocessing import LabelEncoder
      from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import accuracy_score, confusion_matrix
      df = pd.DataFrame({
          "tenure":[1,12,24,36,6,18],
          "monthly_charges":[50,70,80,90,60,75],
          "contract_type":["Month","Year","Year","TwoYear","Month","Year"],
          "churn":[1,0,0,0,1,0]
      })
      df["contract_type"] = LabelEncoder().fit_transform(df["contract_type"])
      X = df[["tenure","monthly_charges","contract_type"]]
      y = df["churn"]
      model = LogisticRegression()
      model.fit(X,y)
      pred = model.predict(X)
      print("Accuracy:", accuracy_score(y,pred))
      print(confusion_matrix(y,pred))

      Accuracy: 1.0
      [[4 0]
       [0 2]]
```

Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.

Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

```
[23]: from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import mean_squared_error
      df = pd.DataFrame({
          "area":[800,1000,1200,1500,1800],
          "bedrooms":[2,3,3,4,4],
          "location_score":[6,7,8,9,9],
          "price":[50,65,75,90,105]
      })
      X = df[["area","bedrooms","location_score"]]
      y = df["price"]
      X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)
      model = LinearRegression()
      model.fit(X_train,y_train)
      pred = model.predict(X_test)
      print("MSE:", mean_squared_error(y_test,pred))

      MSE: 25.000000000070486
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.

Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

```
[25]: from sklearn.preprocessing import StandardScaler
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.metrics import classification_report
      df = pd.DataFrame({
          "glucose":[90,120,150,130,110],
          "BMI":[22,28,35,30,25],
          "age":[25,45,50,40,30],
          "outcome":[0,1,1,1,0]
      })
      X = df[["glucose","BMI","age"]]
      y = df["outcome"]
      X = StandardScaler().fit_transform(X)
      model = DecisionTreeClassifier()
      model.fit(X,y)
      pred = model.predict(X)
      print(classification_report(y,pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         3

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

Q4. Email Spam Detection

You are given email text and spam labels.

Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

```
[27]: from sklearn.feature_extraction.text import TfidfVectorizer
      from sklearn.naive_bayes import MultinomialNB
      df = pd.DataFrame({
          "email_text":["win money now","Meeting tomorrow","Free prize","Project update"],
          "spam":[1,0,1,0]
      })
      vec = TfidfVectorizer()
      X = vec.fit_transform(df["email_text"])
      y = df["spam"]
      model = MultinomialNB()
      model.fit(X,y)
      test = vec.transform(["win free cash"])
      print("Prediction:", model.predict(test))
```

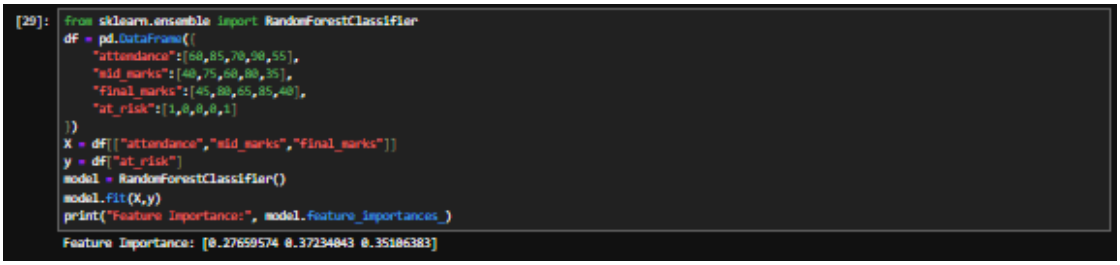
```
Prediction: [1]
```

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.

Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance



```
[29]: from sklearn.ensemble import RandomForestClassifier
      df = pd.DataFrame({
          "attendance":[60,85,70,90,55],
          "mid_marks":[40,75,60,80,35],
          "final_marks":[45,80,65,85,40],
          "at_risk":[1,0,0,0,1]
      })
      X = df[["attendance","mid_marks","final_marks"]]
      y = df["at_risk"]
      model = RandomForestClassifier()
      model.fit(X,y)
      print("Feature Importance:", model.feature_importances_)

      Feature Importance: [0.27659574 0.37234043 0.35106383]
```